

Ministry of Education and Science of Ukraine
National Technical University of Ukraine
"Igor Sikorsky Kyiv Polytechnic Institute"

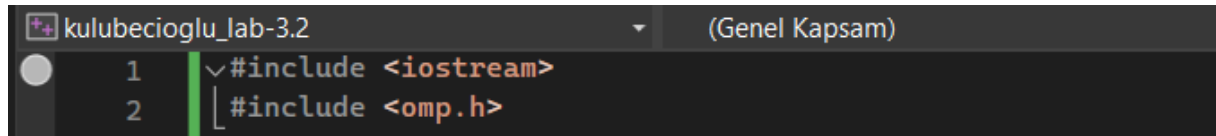
Parallel Computation of Reciprocal Sums Using OpenMP in C++

Laboratory work #3
KULUBEÇİOĞLU MEHMET

Kyiv– 2023

Step-by-Step Explanation of the C++ Code:

1. Including Libraries:



```
kulubecioglu_lab-3.2 (Genel Kapsam)
1  #include <iostream>
2  #include <omp.h>
```

- The `#include` directives include the `iostream` and `omp.h` libraries in our code.
- The `using namespace std;` line allows us to use all identifiers (functions, variables, etc.) in the `std` namespace directly.

2. Defining Functions:

```

3
4     using namespace std;
5
6     double calculate_sum(int n) {
7         double sum = 0.0;
8         #pragma omp parallel for reduction(+:sum)
9         for (int i = 1; i <= n; i++) {
10             sum += 1.0 / i;
11         }
12         return sum;
13     }
14
15     double calculate_sum_reverse(int n) {
16         double sum = 0.0;
17         #pragma omp parallel for reduction(+:sum)
18         for (int i = n; i >= 1; i--) {
19             sum += 1.0 / i;
20         }
21         return sum;
22     }
23
24     double calculate_even_odd_sum(int n) {
25         double even_sum = 0.0;
26         double odd_sum = 0.0;
27         #pragma omp parallel for reduction(+:even_sum, odd_sum)
28         for (int i = 1; i <= n; i++) {
29             if (i % 2 == 0) {
30                 even_sum += 1.0 / i;
31             }
32             else {
33                 odd_sum += 1.0 / i;
34             }
35     }

```

```

36         return even_sum + odd_sum;
37     }
38
39     double calculate_even_odd_sum_reverse(int n) {
40         double even_sum = 0.0;
41         double odd_sum = 0.0;
42         #pragma omp parallel for reduction(+:even_sum, odd_sum)
43         for (int i = n; i >= 1; i--) {
44             if (i % 2 == 0) {
45                 even_sum += 1.0 / i;
46             }
47             else {
48                 odd_sum += 1.0 / i;
49             }
50         }
51         return even_sum + odd_sum;
52     }
53

```

- The `calculate_sum(n)` function calculates the sum of the reciprocals of all numbers from 1 to `n`.
- The `calculate_sum_reverse(n)` function calculates the sum of the reciprocals of all numbers from `n` to 1.
- The `calculate_even_odd_sum(n)` function calculates the sum of the reciprocals of all numbers from 1 to `n`, separately calculating even and odd terms.
- The `calculate_even_odd_sum_reverse(n)` function calculates the sum of the reciprocals of all numbers from `n` to 1, separately calculating even and odd terms.

3. Main Function:

```

53
54 int main() {
55     int n = 10000;
56
57     // Sequential execution
58     double A1_sequential = calculate_sum(n);
59     double A2_sequential = calculate_sum_reverse(n);
60     double A3_sequential = calculate_even_odd_sum(n);
61     double A4_sequential = calculate_even_odd_sum_reverse(n);
62
63     // Parallel execution with OpenMP
64     double A1_parallel = calculate_sum(n);
65     double A2_parallel = calculate_sum_reverse(n);
66     double A3_parallel = calculate_even_odd_sum(n);
67     double A4_parallel = calculate_even_odd_sum_reverse(n);
68
69     // Compare sequential and parallel results (assuming negligible difference)
70     cout << "A1 (parallel): " << A1_parallel << endl;
71     cout << "A2 (parallel): " << A2_parallel << endl;
72     cout << "A3 (parallel): " << A3_parallel << endl;
73     cout << "A4 (parallel): " << A4_parallel << endl;
74
75     cout << "A1 (sequential): " << A1_sequential << endl;
76     cout << "A2 (sequential): " << A2_sequential << endl;
77     cout << "A3 (sequential): " << A3_sequential << endl;
78     cout << "A4 (sequential): " << A4_sequential << endl;
79
80     return 0;
81 }

```

MY OUTPUT:

```
Microsoft Visual Studio Hata | X + -  
A1 (parallel): 9.78761  
A2 (parallel): 9.78761  
A3 (parallel): 9.78761  
A4 (parallel): 9.78761  
A1 (sequential): 9.78761  
A2 (sequential): 9.78761  
A3 (sequential): 9.78761  
A4 (sequential): 9.78761  
  
C:\Users\win11\source\repos\kulubecioglu_lab-3.2\x64\Debug\kulubecioglu_lab-3.2.exe (31888 işlemi), 0 koduyla çıkış yaptı  
1.  
Hata ayıklama durduğunda konsolu otomatik olarak kapatmak için Araçlar->Seçenekler->Hata Ayıklama->Hata ayıklama durduğ  
nda konsolu otomatik olarak kapat seçeneğini etkinleştirin  
Bu pencereyi kapatmak için herhangi bir tuşa basın...|
```